

Software Engineer / 김경우

# 기능을 넘어, 계속 운영되는 구조까지 봅니다.

사내 시스템의 저장 구조와 업무 자동화를 다시 설계하고,  
사용자가 체감하는 성능 문제를 직접 측정해 고쳐온 개발자입니다.

## SELECTED WORK

### 01 CRM 개발 인턴

저장소 · 인증 구조 설계와 RPA 리팩토링

### 02 동아리 모집 사이트

인증 흐름과 API 경계 설계

### 03 축제 서비스 Cardinal

지도 페이지 성능 최적화

— Software Engineer / 김경우

# 만드는 데서 끝내지 않고 실제로 계속 운영되는 과정까지 경험했습니다.

프론트엔드로 시작해 백엔드와 클라우드까지 서비스 전반을 다뤘고,  
기능 구현뿐 아니라 반복해서 손이 가는 지점을 찾아 구조를 바꿨습니다.  
당장 동작하는 것보다 오래 운영될 때 생길 문제를 먼저 봤습니다.

## WAY 01

### 분리

Separate Concerns

프론트엔드가 내부 시스템을 직접 알지 않도록, 인증과 저장소와 외부 연동을 각각 필요한 계층에서 처리합니다.

CRM 인턴

Azure Function, Blob Storage 연동 구조 설계

React / TypeScript / Azure Function App / Blob Storage / Dataverse

## WAY 02

### 수명

Decide the Lifetime

기능을 만들기 전에 데이터가 어디에 얼마나 남아 있어야 하는지부터 정하고, 필요한 기간에 맞춰 관리합니다.

동아리 모집 사이트

Redis TTL 기반 인증 흐름, 지원서 API 설계

Java / Spring Boot / Spring Security / JPA / Redis / PostgreSQL

## WAY 03

### 측정

Measure Bottlenecks

느리다는 느낌에 의존하지 않고 직접 측정하며, 여러 시도를 비교해 실제로 느려지는 지점을 찾습니다.

Cardinal

지도 페이지 INP 1130ms → 34ms 개선

React / TypeScript / Mapbox / framer-motion

흩어진 책임과 흐름을 다시 구조화했습니다.

# 01 / CRM 개발 인턴

Azure · Dataverse · Power Platform 기반 고객사 포털 개발 및 사내 자동화 리팩토링

고객사 포털에서는 프론트엔드가 내부 시스템을 직접 알지 않도록 **API · 인증 · 저장소 계층을 분리**했고, 고객사 업무 자동화에서는 하드코딩된 흐름을 **마스터 데이터 기반 구조**로 재정리했습니다.



## TOPIC 01

### Blob Storage 저장소 구조

Dataverse에 쌓이던 이미지 바이너리를 Blob으로 분리하고, 임시 업로드 · 커밋 흐름을 설계

## TOPIC 02

### 프론트엔드 인증 구조

Dataverse 토큰을 프론트에서 걷어내고 JWT 기반으로 인증 책임을 백엔드로 이전

## TOPIC 03

### Power Automate 리팩토링

하드코딩된 184개 액션을 마스터 데이터 기반 41개 반복 구조로 재정리

Pyro Engineering

# 고객 포털과 CRM을 연동하는 문의 응대 서비스

BACKGROUND

고객이 문의를 남기고 처리 상태를 확인하는 웹 포털입니다. 포털 문의는 내부 CRM(Dataverse)의 Service Case로 그대로 이어져야 했습니다.

무엇을 만들었나

|          |                           |
|----------|---------------------------|
| 로그인      | 계정 인증 · 내 정보              |
| 문의 등록·조회 | Service Case 생성 · 목록 · 상세 |
| 파일·이미지   | 본문 이미지 · 첨부 업로드           |
| CRM 연동   | 내부 CRM에서도 확인              |

ROLE 단독 수행 · FE · API · Cloud · Storage

STACK React · TypeScript · Azure Function App · Blob Storage · Dataverse

PROBLEM

기능은 동작했지만, 운영 관점에서 구조적 리스크가 있었습니다.

이미지가 Dataverse에 계속 쌓인다

1 리치 텍스트 이미지가 Dataverse에 바이너리로 저장되어, 문의가 누적될수록 저장 용량을 잠식했습니다.

RISK 용량 한도 초과 시 추가 구매 · 증가 속도 예측 불가

프론트가 내부 CRM에 직접 의존한다

2 프론트엔드가 Dataverse 토큰 · URL · 응답 구조를 직접 알고 있어 내부 시스템과 강하게 묶여 있었습니다.

RISK 내부 변경이 곧 화면 코드 변경 · 토큰 노출 위험

책임이 섞여 유지보수가 어렵다

3 이미지 · 파일 · 인증 책임이 프론트와 Dataverse에 흩어져 시스템 경계가 분명하지 않았습니다.

RISK 운영 이후 구조 변경 시 마이그레이션 비용 급증

그래서 이미지 저장 책임과 내부 시스템 접근 책임을 분리하는 방향으로 재설계했습니다.

# 이미지 바이너리는 Blob Storage로 옮기고, Dataverse에는 HTML·메타데이터만 남겼습니다.



## 무엇이 달라졌나

| 구분           | BEFORE       | AFTER              |
|--------------|--------------|--------------------|
| 이미지 저장 위치    | Dataverse 첨부 | Azure Blob Storage |
| Dataverse 역할 | 데이터 + 파일     | 데이터 + 메타데이터        |
| 용량 부담        | 높음 · 예측 어려움  | 분리로 완화             |
| 이미지 조회       | Dataverse 직접 | 백엔드 권한 검증 후        |
| 운영 확장성       | 용량 증가에 취약    | 저장소 분리로 우수         |

## 왜 운영 전에 바꿨나

- 운영이 시작되면 구조는 바꾸기 어렵습니다. 이미지가 쌓이기 전에 미리 분리했습니다.
- 1

기존 이미지를 Blob Storage로 마이그레이션
- 2

본문 HTML 안의 이미지 경로를 전부 치환
- 3

이미지와 Service Case 연결 정보 재구성
- 4

CRM 화면에서 깨지는 이미지 검증
- 5

권한과 접근 정책 재설계

용량 부담 완화

바이너리를 Blob으로 분리

백엔드가 정책 제어

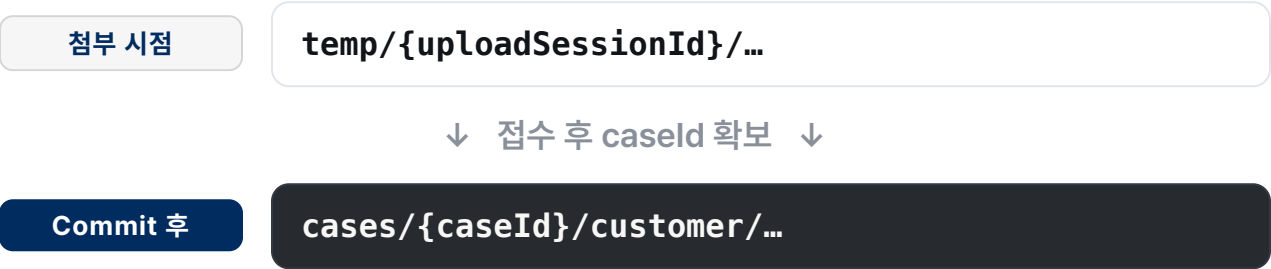
경로·메타·권한을 한 계층에서

운영 확장성 확보

저장소 분리로 안정적 확장

# Temp로 먼저 올리고, Commit으로 문의에 귀속시킵니다.

작성 시점엔 최종 경로를 정할 수 없다



최종 경로는 **caseId** 기준인데, 이미지를 첨부하는 작성 중에는 아직 Service Case가 없어 caseId가 없습니다.

그래서 TEMP 단계가 필요했다

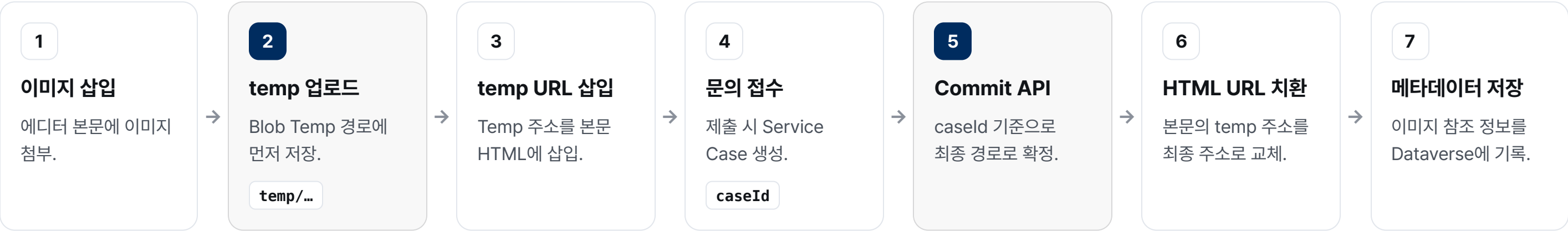
- 1

**접수 전에 이탈할 수 있다**  
이미지만 올리고 제출을 취소하면 **Case에 연결되지 않은 이미지**가 남는다 — temp로 분리 관리.
- 2

**한 문의에 여러 이미지**  
uploadSessionId로 묶어, Commit 시 어떤 이미지들이 이 문의에 속하는지 추적한다.
- 3

**최종 경로는 caseId 이후**  
caseId가 생긴 뒤에야 최종 경로가 정해지므로, 그전까지 **temp에 안전하게 보관**한다.

TEMP 업로드 → COMMIT · 전체 순서



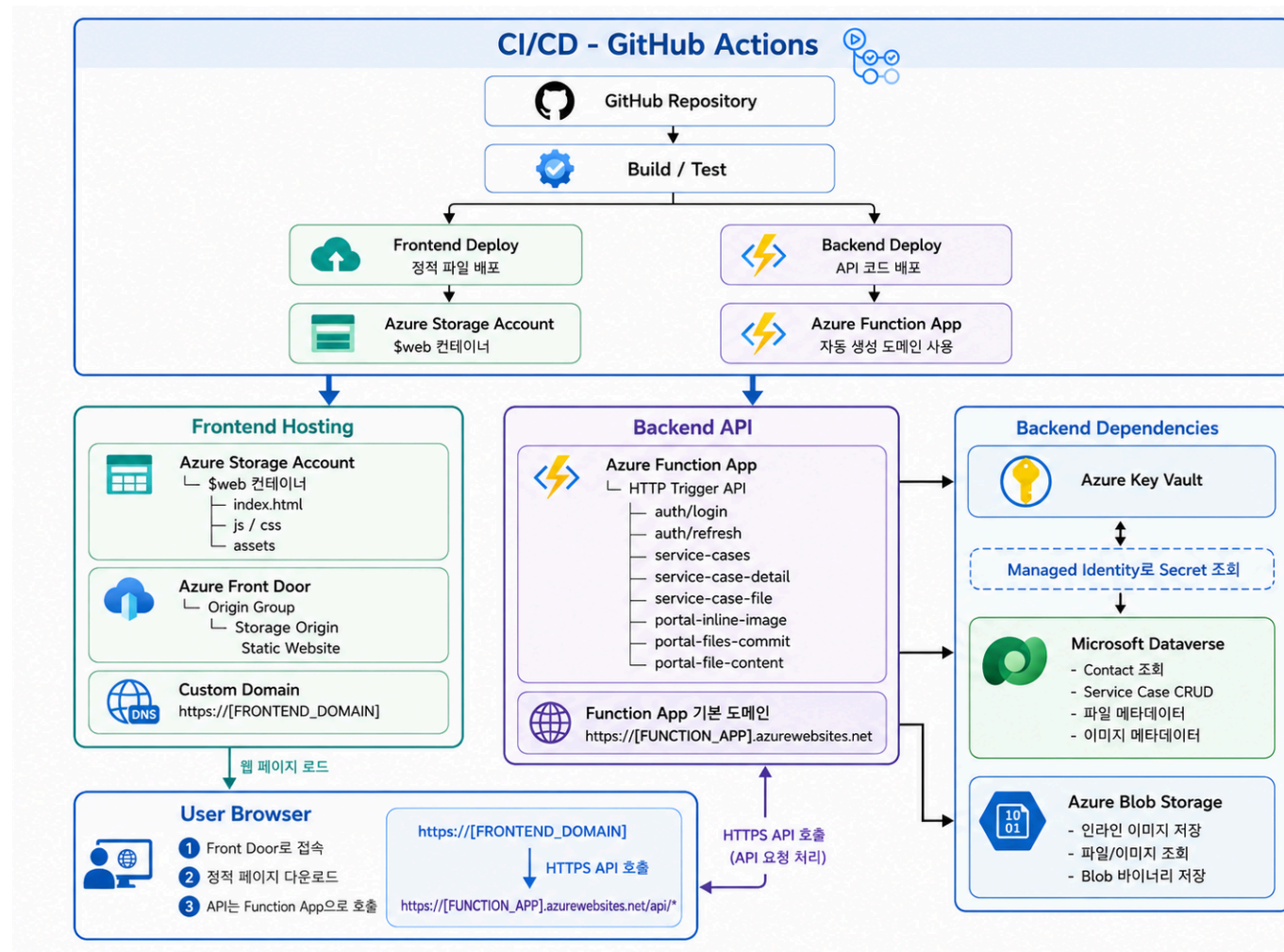
Commit은 단순 후처리가 아니라, Temp 이미지를 실제 문의에 귀속시키는 단계입니다.

작성 중 흩어져 있던 Temp 이미지가 Commit을 거쳐 하나의 Service Case로 묶입니다.

## — Solution · Azure Functions 백엔드 위임 &amp; 인증 구조

프론트는 우리 API만 호출하고,  
내부 시스템 접근은 백엔드 안에서 처리합니다.

## 전체 시스템 아키텍처



## 이 구조가 보장하는 것

## 1 백엔드 위임

프론트는 API·JWT만, Dataverse·Blob 접근은 백엔드 안에서만.

## 2 두 토큰 분리

프론트는 Pyro JWT만, Dataverse Token은 백엔드 내부에서만.

## 3 Secret은 Key Vault에

민감 정보는 Key Vault에 두고 Managed Identity로 접근.

## 4 소유권 검증

JWT `contactId == 리소스 owner` 일 때만 케이스·파일을 반환한다.

## 5 CI/CD 자동 배포

GitHub Actions로 Storage·Function App에 자동 배포.

프론트는 내부 CRM 구조를 몰라도 되고,  
인증·저장소 접근 정보는 백엔드 안으로 숨겨졌습니다.

— Power Automate RPA

# 폴더 생성 자동화를 데이터 기반 구조로 리팩토링

## BACKGROUND

업무 프로젝트가 상위 프로젝트에 연결되면, SharePoint에 폴더 트리를 만들고 Dataverse에 폴더 메타데이터 레코드를 생성하는 사내 자동화입니다.

### 자동화가 하는 일

폴더 트리    레벨2·레벨3·하위 폴더 생성

산출물 레코드    Dataverse에 메타데이터 저장

권한 부여    계열사별 폴더 접근 권한 설정

184 → 41 actions    약 77.7% 감소

ROLE    단독 수행 · Flow 분석 · 리팩토링

STACK    Power Automate · SharePoint · Dataverse

## PROBLEM

동작은 했지만, 업무 규칙이 Flow 안에 하드코딩되어 바꾸기 어려운 구조였습니다.

- 1

폴더 하나 = 액션 하나    연결 액션 126개

레벨2·레벨3·하위 폴더가 전부 개별 액션 — 폴더 하나만 추가·변경해도 Flow를 직접 수정해야 했습니다.
- 2

분기·반복이 거의 없는 절차형    Foreach 2개

같은 패턴을 묶지 않고 직렬로 나열 — 액션 수가 폴더 수만큼 무한정 늘어나는 구조였습니다.
- 3

긴 if 식 + 문자열 payload

경로는 긴 if 중첩식, 레코드는 replace 연쇄로 조립 — 이름 하나 바뀌면 줄줄이 깨졌습니다.

## LEGACY 184 ACTIONS · 구성

대부분이 단순 폴더 생성 액션

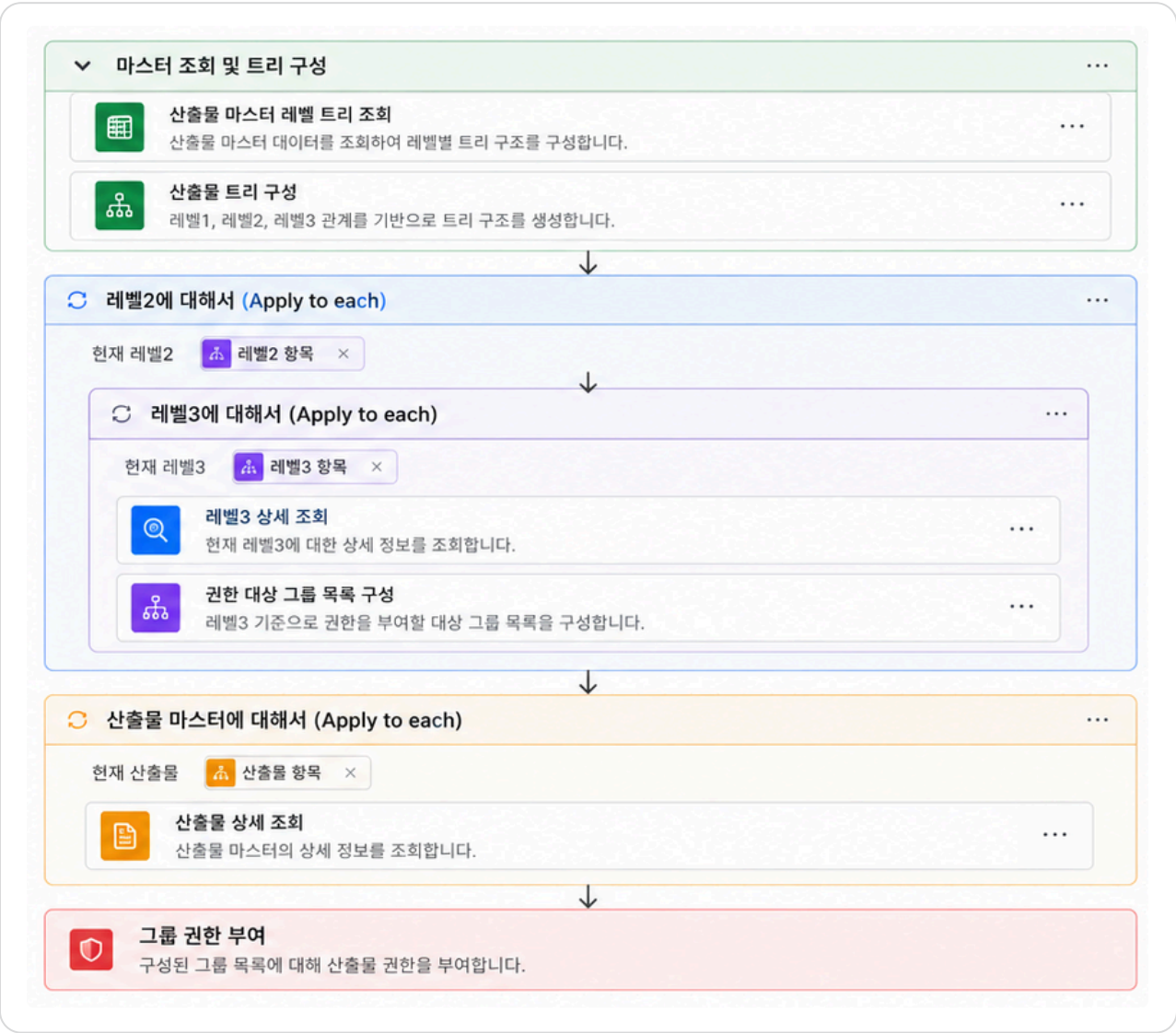


그래서 업무 규칙을 Flow에서 **Dataverse** 마스터 데이터로 옮기는 방향으로 리팩토링했습니다.

— Power Automate RPA 리팩토링 · 해결

업무 규칙을 **Dataverse** 마스터 데이터로 옮기고,  
데이터에서 트리를 만들어 반복 생성하도록 바꿨습니다.

REFACTORED FLOW · 실제 화면



**AFTER** 개별 액션 나열이 **그룹화된 반복 구조**로 정돈

핵심 개선

- 1 폴더 구조를 데이터로 관리**  
폴더가 바뀌어도 자동화를 고칠 필요 없이 **데이터만 수정**하면 된다.
- 2 중복 폴더 자동 정리**  
같은 폴더가 여러 번 나와도 **한 번만** 만들어지도록 데이터를 정리.
- 3 데이터를 안전하게 조립·검증**  
문자열 조립을 **구조화된 방식**으로 바꾸고, 쓸 수 없는 문자를 미리 걸러 오류를 방지.

|                     |                  |
|---------------------|------------------|
| <b>41</b> actions   | 데이터 가공 액션이 새로 등장 |
| OpenApiConnection   | 24               |
| Foreach             | 4                |
| Compose · 신규        | 5                |
| Query · Select · 신규 | 5                |

핵심은 액션 수가 아니라 **수정 지점의 이동**  
폴더가 바뀌면 Flow가 아니라 마스터 데이터만 고치면 됩니다.

인증에서 중요한 건 기능보다 인증 상태를 언제까지 유지할지 정하는 것이었습니다.

# 02 / 동아리 부원 공식 모집 사이트

Java 17 · Spring Boot 3.4 · PostgreSQL · Redis · Docker Compose · 약 2,700명 방문

지원자를 받는 사이트는 모집 기간에만 운영되고, 특정 시간대에 방문이 몰립니다.  
외부 인증 API에 의존하던 본인 확인을 직접 관리하는 인증 방식으로 바꾸고, 인증번호의 유효시간을 Redis에서 관리했습니다.

01

인증 과정을 직접 관리

외부 인증 API에 맡기던 본인 확인을 메일 발송 + Redis 검증 방식으로 바꾸고, 인증번호 만료와 재발급 기준을 직접 설정했습니다

외부 API 의존 → 자체 인증 / 인증 정책 직접 관리

02

인증번호는 TTL로 자동 만료

인증번호는 일정 시간이 지나면 사용할 수 없어야 하는 데이터이므로 별도 삭제 로직 대신 Redis TTL을 적용해 자동으로 만료되도록 했습니다

TTL · 30분 / 재발급 60초 · 최대 3회

— 동아리 부원 공식 모집 사이트

# 인증 정책을 직접 제어할 수 없었습니다

## BACKGROUND

지원자가 본인 확인 후 지원서를 제출하고 결과를 조회하는 사이트입니다. 모집 기간에만 운영되며 짧은 기간에 접속이 몰립니다.

### 사용자 요청 흐름

|       |                                            |
|-------|--------------------------------------------|
| 본인 확인 | 인증번호 발송 → 입력 → 검증                          |
| 지원서   | 생성 / 조회 / 수정                               |
| 결과 조회 | 서류 / 최종 결과 확인                              |
| 지원 상태 | <div>지원함   서류합격   서류탈락   최종합격   최종탈락</div> |

개발 시점의 조건

개발 기간

2주

백엔드 4인

지원 규모

예측 불가

모집 공고 후 결정

|       |                                                      |
|-------|------------------------------------------------------|
| ROLE  | 백엔드 · 인증 흐름 · 지원서 API                                |
| STACK | Java 17 · Spring Boot 3.4 · JPA · PostgreSQL · Redis |

## PROBLEM

인증은 됐지만, 필요한 정책을 직접 적용하기 어려웠습니다.

- 1

만료 시간을 정할 수 없음

인증

발급과 검증이 외부 API에서 처리돼, 인증번호를 몇 분 유지할지 서비스에서 정할 수 없었습니다.
- 2

재발송을 제한할 수 없음

인증

같은 이메일로 계속 요청할 수 있었고, 요청한 만큼 메일도 반복 발송됐습니다.
- 3

비밀번호가 평문으로 저장

재설정 시 입력값을 그대로 저장했습니다. 지원서에는 학번·연락처도 함께 저장됩니다.
- 4

실패 원인을 구분하기 어려움

만료·불일치·중복의 처리 방식이 일정하지 않아 프론트에서 원인을 구분하기 어려웠습니다.

정리한 질문

Q1

인증번호를 어디에 저장해야 시간이 지나면 자동으로 삭제되는가

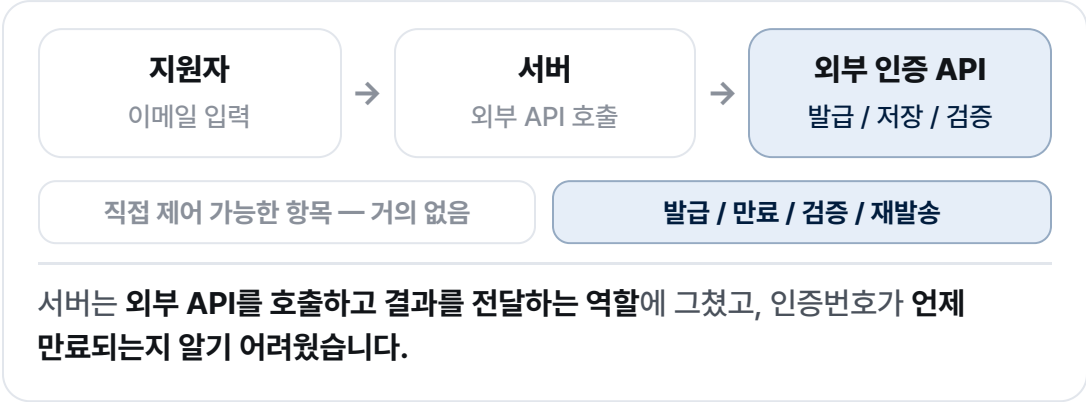
Q2

만료, 불일치, 미발급을 어떻게 구분해 전달할 것인가

인증번호는 일정 시간이 지나면 반드시 사라져야 하는 데이터였습니다.

## 기존 인증 구조

외부 API 위임



기존 인증 처리 AuthService.java

BEFORE

```
// 발급과 검증이 외부 API 호출로 끝남
Map<String, Object> result =
    UnivCert.certifyCode(key, email, univName, code);

if ((boolean) result.get("success")) { ... }

// 재설정 시 입력값을 그대로 저장
app.updatePassword(requestDto.getPassword()); // 평문
```

지원이 몰릴 때 반복되던 상황

1

인증번호 요청 → 메일 도착 지연

발송

2

다시 요청 → 이전 인증번호도 그대로 유효

발송

재발송 기준이 없어 요청할 때마다 새로운 메일이 발송됐습니다.

이 구조에서 확인하기 어려웠던 것

•

인증번호가 언제 만료되는지 — 서비스에서 알 수 없었습니다.

•

한 사용자가 몇 번 요청했는지 — 별도로 관리하지 않았습니다.

— 해결 과정

인증번호의 만료, 횟수, 검증을 직접 관리했습니다

외부 API에 맡기던 인증을 서비스에서 처리하도록 바꾸고,  
만료 시간과 재발송 기준을 직접 설정했습니다.

AFTER — 인증 흐름

/auth

STEP 01

발송 요청

이메일과 요청 유형을 확인합니다.

POST /send-code/{signup,reset}

STEP 02

횟수 제한

이메일별 요청을 기록해 60초 동안 3회를 넘으면 차단합니다.

email\_limit\_{email} · TTL 60s

STEP 03

저장

SecureRandom으로 6자리 인증번호를 생성해 Redis에 30분간 저장합니다.

SET {email} = code · TTL 30m

STEP 04

검증

값이 없으면 만료, 다르면 불일치로 구분합니다.

GET {email} → null? / equals?

STEP 05

사용

인증 후 지원서 생성 또는 비밀번호 재설정으로 이어집니다.

/application · /reset-password

1 Redis TTL로 만료 처리

발급 시간을 저장해 매번 계산하는 대신, 30분 뒤 값이 자동으로 삭제되도록 했습니다.

MailService.java

```
// 코드 생성 + 수명을 함께 지정
String authCode = createCode(); // SecureRandom 6자리
redisUtil.setDataExpire(email, authCode, 60 * 30L);

// 검증 - 값이 없으면 곧 만료
String found = redisUtil.getData(email);
if (found == null) throw new VerificationFailedException(...);
```

인증번호를 저장할 때 만료 시간까지 함께 지정해, 시간이 지나면 Redis에서 값이 자동으로 사라집니다. 검증 시에는 별도의 시간 계산 없이 값이 남아 있지만 확인했습니다.

결과  
별도 삭제 로직 없이 인증번호 만료를 처리했습니다.

2 실패 원인을 구분

만료와 불일치는 사용자가 해야 할 행동이 다르기 때문에 저장된 값의 상태에 따라 결과를 나눴습니다.

저장된 값의 상태 → 결과

|      |               |     |
|------|---------------|-----|
| 없음   | TTL로 삭제됨      | 만료  |
| 값 다름 | 저장된 값과 입력이 다름 | 불일치 |
| 값 같음 | 유효 시간 안에 같은 값 | 통과  |

재발송

기존 번호를 삭제한 뒤 새 번호를 저장해, 유효한 인증번호가 하나만 남도록 했습니다.

결과  
프론트에서 만료, 불일치, 횟수 초과를 구분해 안내할 수 있게 됐습니다.

3 기능별 책임 분리

MailService

인증번호 발급 / 저장 / 검증 / 재발송 제한

PasswordService

BCrypt 해싱 / 비밀번호 비교

ApplicationService

지원서 생성 / 조회 / 수정

ResultService

지원 상태에 따른 결과 조회

비밀번호도 평문 저장에서 BCrypt 해싱 방식으로 변경하고, 기존과 같은 값은 사용할 수 없도록 했습니다.

결과  
인증 관련 변경 사항을 MailService 중심으로 관리할 수 있게 됐습니다.

— 결과

# 인증 흐름을 API별로 나눠 구현했습니다

## 구현한 API

|      |                                  |                                |
|------|----------------------------------|--------------------------------|
| POST | /auth/send-code/{signup,reset}   | 가입 여부 확인 후 인증번호 발송 / 60초 3회 제한 |
| POST | /auth/verify-code                | Redis 값으로 만료 / 불일치 / 통과 구분     |
| POST | /auth/reset-password             | BCrypt 저장 / 기존 비밀번호와 같으면 거절    |
| POST | /application · /application/view | 지원서 생성 / 조회 / 조회 전 비밀번호 확인     |
| PUT  | /application                     | 이메일, 비밀번호, 지원 상태는 기존 값 유지      |
| POST | /result/{document,final}         | 지원 상태에 따라 조회 가능한 결과만 반환        |

|                                                         |                                     |                        |
|---------------------------------------------------------|-------------------------------------|------------------------|
| 예외 처리 표준화                                               |                                     | GlobalExceptionHandler |
| 400                                                     | VerificationFailed / DuplicateEmail |                        |
| 401                                                     | InvalidPassword                     |                        |
| 403                                                     | InvalidUserStatus                   |                        |
| 429                                                     | EmailLimitExceeded                  |                        |
| 모든 API 결과를 success / code / message / data 형식으로 통일했습니다. |                                     |                        |

|           |        |                 |
|-----------|--------|-----------------|
| 운영 결과     |        |                 |
| 총 방문자     | 2,700명 | 모집 기간 집중        |
| 인증번호 유효시간 | 30분    | Redis TTL 자동 만료 |

## WHY / REDIS를 선택한 이유

### 인증번호는 일정 시간이 지나면 없어져야 하는 데이터입니다

만료 시간을 매번 계산하거나 별도 삭제 작업을 두는 대신, TTL로 자동 삭제되는 Redis를 선택했습니다.

| 방식                         | 특징                                         |    |
|----------------------------|--------------------------------------------|----|
| Redis TTL<br>setDataExpire | 시간이 지나면 자동 삭제되어 별도 만료 계산이나 삭제 작업이 필요 없습니다. | 채택 |
| DB 테이블<br>created_at 비교    | 관리하기 쉽지만 만료된 값이 남아 별도 정리가 필요합니다.           | 대안 |
| 외부 인증 API<br>UnivCert      | 적용은 빠르지만 만료와 재발송 기준을 직접 정하기 어렵습니다.         | 이전 |

### 더 다뤄 볼 부분

- 재발송 제한이 이메일 기준이라 다른 주소로 우회할 수 있습니다.
- 지원서 수정 시 실제 인증 완료 여부를 서버가 다시 확인하는 절차가 필요합니다.
- 동일 지원서에 수정 요청이 동시에 들어오는 상황은 별도로 검증하지 못했습니다.
- 인증 완료 여부를 짧은 유효시간의 토큰으로 남기면 이후 요청에서도 확인할 수 있습니다.

회고  
처음에는 인증을 메일을 보내고 값을 확인하는 기능으로 생각했습니다. 직접 구현하면서 인증번호를 어디에 저장하고, 얼마나 유지하고, 언제 없앨지까지 함께 설계해야 한다는 것을 배웠습니다.

체감이 아니라 **측정값으로** 병목을 확인했습니다.

# 03 / 학교 축제 서비스 Cardinal

React · TypeScript · Mapbox 기반 축제 지도 페이지 성능 개선 · 프론트엔드

사용자가 자주 이용하는 지도 페이지에서 **리스트를 여닫을 때 심한 프레임 드롭**이 발생했습니다.

가설을 세우고 측정한 뒤, 효과가 부족하면 다시 원인을 분석하며 **병목 지점을 좁혀**갔습니다.

## 01 첫 시도 — IntersectionObserver

화면 근처에 들어온 항목만 이미지를 만들도록 바꿨지만, 닫을 때는 여전히 프레임이 끊겼습니다

INP 1130 → **343** ms

## 02 실제 병목 — 원본 이미지 해상도

이미지 개수가 아니라 **표시 크기에 비해 지나치게 큰 원본 해상도**의 처리 비용이 문제였습니다

343 → **34** ms

— 학교 축제 지도 페이지

마커·리스트를 열 때  
끊기는 지도 화면

BACKGROUND

동아리에서 개발한 학교 축제 웹 서비스로, Mapbox 지도 위에 9개 카테고리 마커와 부스 리스트를 구현했습니다.

지도 페이지 주요 기능

|       |                           |
|-------|---------------------------|
| 카테고리  | 선택 시 해당 마커만 지도에 표시        |
| 마커 클릭 | 주점·푸드트럭 등 부스 리스트를 드로어로 표시 |
| 드로어   | 높이를 20%→80%로 조절하는 인터랙션    |

INTERACTION (INP)

1130 → 343 → 34ms

약 97% 단축

ROLE 프론트엔드 · 지도 페이지 · 성능 측정·개선

STACK React · TypeScript · Mapbox · framer-motion

PROBLEM

기능은 동작했지만, 리스트를 여닫을 때 화면이 끊겼습니다.

- 1

드로어 애니메이션 프레임 드롭

리스트를 열고 닫는 과정에서 심한 프레임 드롭이 발생했습니다.
- 2

사용 빈도가 높은 화면

지도와 부스 목록은 사용자가 반복해서 조작하는 화면이라 지연이 바로 체감됐습니다.
- 3

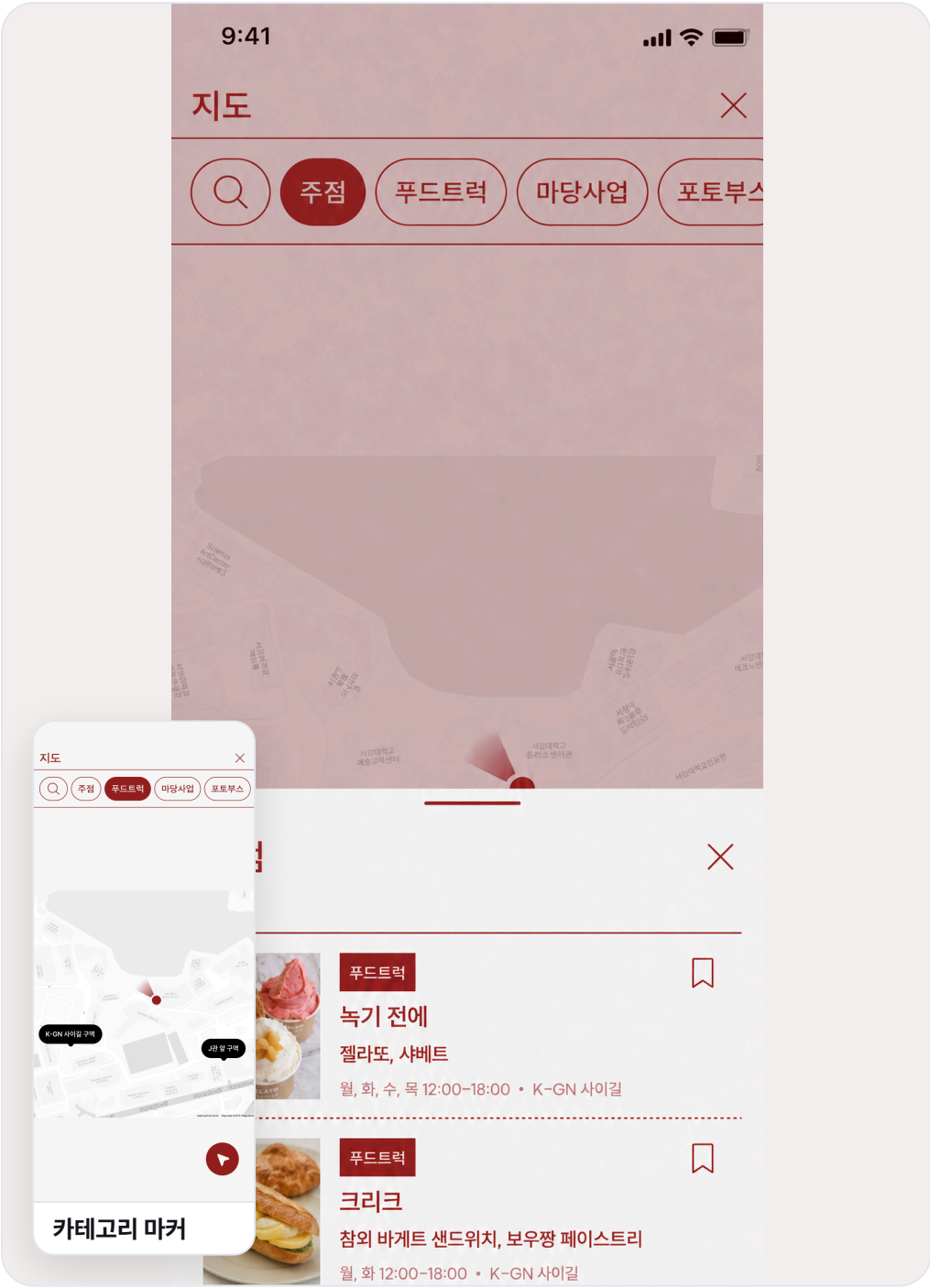
일반 지표만으로 원인 확인 어려움

Lighthouse의 LCP·TBT만으로는 해당 인터랙션 지연을 설명하기 어려웠습니다.

Performance 탭과 Clarity를 추가로 확인하며 병목을 좁혔습니다.

문제가 발생한 화면

마커 → 부스 리스트 드로어



마커 클릭 → 부스 리스트 드로어 · 여닫을 때 프레임 드롭 발생

## — 병목 추적

## 병목 원인을 단계별로 확인했습니다

## 1 Lazy Load

개선 제한

이미지 요청 시점 늦추기

## 가설

한 번에 많은 이미지를 불러오는 것이 원인이라고 보고  
**lazy loading**을 먼저 적용했습니다.

리스트의 썸네일 &lt;img&gt;

```
<img
  src={src}
  loading="lazy"
  decoding="async"
  fetchPriority="low"
/> // 펼치면 전체 항목은 DOM에 마운트
```

- 기대 — 화면 밖 이미지 요청을 미뤄 초기 처리량 감소
- 실측 — 체감은 일부 개선됐지만 지표 변화는 제한적

## 확인

lazy 는 요청 시점을 조절하지만, 드로어를 열 때 **항목 전체가 DOM에 생성**되는 구조는 그대로였습니다.

## 2 IntersectionObserver

부분 개선

화면 근처 이미지만 렌더링

## 시도

**IntersectionObserver**로 화면 근처에 들어온 항목만  
<img> 를 생성하도록 변경했습니다.

LazyImage.tsx

```
const io = new IntersectionObserver((e) => {
  if (e[0].isIntersecting) {
    setInView(true);
    io.unobserve(e[0].target);
  }
}, { root, rootMargin, threshold });

{inView && !error && <img src={src} ... />}
```

- rootMargin 으로 화면 진입 전에 미리 로드
- 감지 후 unobserve 로 불필요한 재관찰 방지
- 재사용 컴포넌트에 memo + skeleton 적용

## INP 1130 → 343MS

열 때는 개선됐지만 닫을 때는 여전히 프레임 드롭이  
발생해, 이미지 개수 외의 **처리 비용**을 확인했습니다.

## 3 원본 이미지 해상도

주요 병목

Network 탭에서 이미지 크기 확인

## 발견

이미지 **응답 시간이 길어** 원본 파일을 확인한 결과, 실제  
표시 크기에 비해 **해상도가 지나치게 컸습니다.**

# 6000×4000

약 2,400만 픽셀

- 표시 크기 **600×400** 대비 픽셀 수 약 **100배**
- 화면에서 작게 표시되도 **고해상도 원본**을 처리한 뒤 축소

## 개선안

측정 결과를 정리해 업로드 시 **AWS Lambda로 이미지를  
자동 리사이즈**하는 방식을 백엔드에 제안했습니다.

## 병목 확인

**고해상도 이미지 처리 비용**이 드로어 애니메이션과  
겹치면서 응답 지연과 프레임 드롭이 발생했습니다.

## — 개선과 결과

## 프론트와 이미지 처리 구조를 함께 개선했습니다

HOW · 처리 대상 수와 이미지 크기를 각각 줄였습니다

## FRONTEND · 담당

1130 → 343 ms

## 렌더링 대상 수 감소

IntersectionObserver로 화면 근처 항목만 `<img>` 를 생성해 한 번에 처리하는 이미지 수를 줄였습니다.

프론트 개선 이후에도 지연이 남았습니다.

화면에 보이는 이미지 자체가 6000×4000 원본이어서 이미지 처리 비용이 계속 발생했습니다.

## BACKEND · 제안

343 → 34 ms

## 이미지 리사이즈 제안

측정 결과와 원본 이미지 크기를 공유하고, 업로드 시 **AWS Lambda**에서 표시 크기에 맞게 자동 리사이즈하도록 제안했습니다.

프론트에서는 처리할 이미지 수를 줄이고, 백엔드에서는 이미지 한 장의 처리 비용을 줄이는 방식으로 개선했습니다.

## RESULT

INP — 마커 클릭 후 응답

1130 → 34 ms

처리 대상 수와 이미지 크기를 함께 줄여 약 97% 단축

## • 프레임 드롭 개선

반복 드롭 → 정상 동작

긴 리스트를 닫을 때도 이전보다 끊김 없이 동작했습니다.

## • 이미지 처리량 감소

2,400만 → 24만 px

표시 크기에 맞게 리사이즈해 이미지당 픽셀 수를 약 1/100로 축소했습니다.

## • 네트워크 전송량 감소

원본 이미지 전송량 ↓

화면에 필요한 이미지만 표시 크기에 맞는 해상도로 받아 전송량도 줄였습니다.

## • 사용자 행동 확인

MS Clarity

세션 녹화에서 개선 전후를 비교하고 반복 클릭 감소를 확인했습니다.

측정값을 기준으로 원인을 좁히고, 프론트와 이미지 처리 구조를 함께 개선했습니다.

THANK YOU

# 읽어주셔서 감사합니다

궁금한 점이나 함께 이야기하고 싶은 부분이 있다면  
언제든 편하게 연락 주세요.

---

EMAIL

shu07002@gmail.com

PHONE

+82 10-3997-5917

GITHUB

github.com/shu07002

BLOG

shu07002.tistory.com